МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №3

з курсу: Безпека інформаційних систем

виконав студент 4 курсу
групи ДА-11
Сидько Н.С

КИЇВ
2024

Завдання:

Програма повинна реалізувати схему розрахунку ключів шифрування за алгоритмом RSA, сформувати відкриті та закриті (секретні) ключі шифрування. Далі програма має реалізувати режими шифрування та розшифрування файлів із перевіркою достовірності результатів.

Потрібно сформувати вихідний файл із текстом, далі Програма формує зашифрований файл. Після цього Програма повинна забезпечити можливість розшифрування файлу, який повинен збігтися з вихідним.

В процесі роботи Програми необхідно запросити введення з клавіатури довжини бази ключа шифрування, далі вивести ключ на екран (у RSA він складається з двох частин – відкрита і закрита частини). Далі Програма запитує імя файлу для шифрування та імя файлу для запису зашифрованої інформації.

Виконання завдання:

Для виконання завдання потрібно виконати наступний алгоритм:

1)Спочатку ми реалізуємо генерацію ключів шифрування (відкритий та закритий). Ми генеруємо 2 простих числа, p та q, за допомогою функцій

```
def RabinMillerTest(n, k=5):  1 usage
    if n <= 1:
        return False
    if n == 2 or n == 3:
        return True
```

Та

```
def Generate_num(bits):  3 usages
    while True:
        my_number = random.getrandbits(bits) | (1 << bits - 1) | 1
        if RabinMillerTest(my_number):
            return my_number
```

З минулої лабораторної роботи.

Далі ми обраховуємо функцію ейлера, відкриту експоненту e та закриту d. І генеруємо ключі

```
def Gen_Keys(key_size):  1 usage
    p = Generate_num(key_size // 2)
    q = Generate_num(key_size // 2)
    while p == q:
        q = Generate_num(key_size // 2)
    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    if gcd(e, phi) != 1:
        e = random.randrange( start: 3, phi,  step: 2)
        while gcd(e, phi) != 1:
            e = random.randrange( start: 3, phi,  step: 2)

    d = MultiplicateInv(e, phi)

    return (e, n), (d, n), p, q
```

2) Потім ми виконуємо шифрування файлу (читаємо файл, шифруємо дані, зберігаємо зашифровані дані)

```python
def Encrypt_File(input_filename, output_filename, public_key):  1 usage
    with open(input_filename, 'rb') as f:
        file_data = f.read()

    encrypted_data = Encryption(public_key, file_data)
    save_encrypted(output_filename, encrypted_data)

    return input_filename, output_filename
```

3) Потім розшифровуємо файл (зчитали зашифрований файл, виконали розшифрування, зберігли розшифровані дані у файл)

```python
def Decrypt_File(input_filename, output_filename, private_key):  1 usage
    encrypted_data = load_encrypted(input_filename)
    decrypted_data = Decryption(private_key, encrypted_data)

    with open(output_filename, 'wb') as f:
        f.write(decrypted_data)

    return input_filename, output_filename
```

Результат роботи коду:

```
C:\Users\user\PycharmProjects\BisLab3\pythonProject\.venv\Scripts\python.exe C:\Users\use
Введіть довжину ключа (має ділитися на 64, рекомендована 1024): 1024

Компоненти відкритого ключа:
n (модуль): 553451529474372710219860517931206241247278892637261515771347497801973390817798
e (експонента): 65537

Компоненти закритого ключа:
n (модуль): 553451529474372710219860517931206241247278892637261515771347497801973390817798
d (закрита експонента): 229801837436205962898863915253123948865846059881367172241175337755
p (перше просте число): 756833590400496762884905185220038618931877557782677031027430868577
q (друге просте число): 731272417733864685010121682862162060381079292066619309223696716427

Введіть ім'я файлу для шифрування: 1.txt
Введіть ім'я файлу для запису зашифрованої інформації: 2.txt

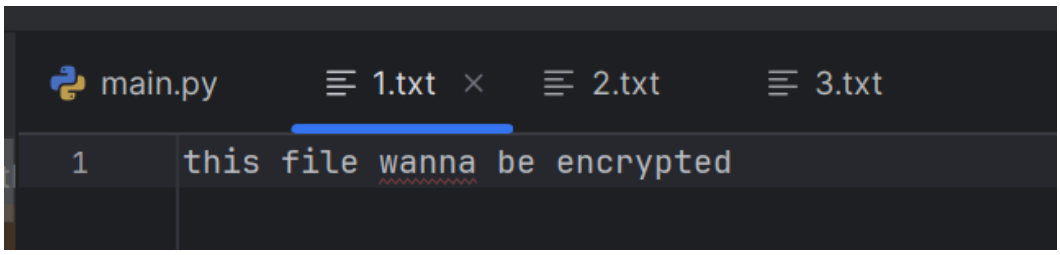
Файл '1.txt' успішно зашифровано у файл '2.txt'.

Введіть ім'я файлу для розшифрування інформації: 3.txt

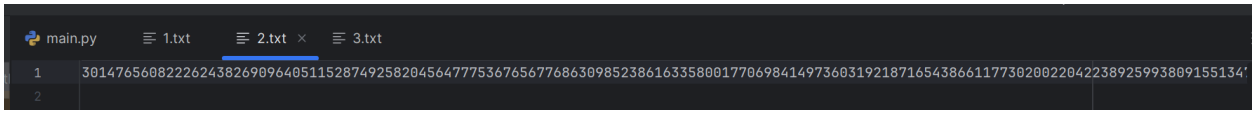
Файл '2.txt' успішно розшифровано у файл '3.txt'.

Process finished with exit code 0
```

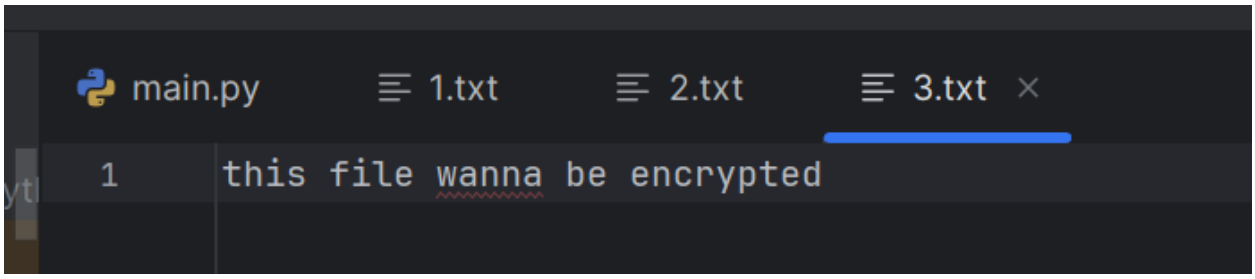
1.txt



2.txt



3.txt



Висновок.

У ході виконання другої лабораторної роботи, було реалізовано схему схему розрахунку ключів шифрування за алгоритмом RSA.Також реалізовано процедуру шифрування за схемою RSA і протестовано на тектових файлах з інформацієїю, яку спочатку було записано в один текстовий файл, зашифровано в інший текстовий файл, і розшифровано в третій текстовий файл.

Лістинг коду.

```
import random
from math import gcd

def RabinMillerTest(n, k=5):
    if n <= 1:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False

    s, d = 0, n - 1
    while d % 2 == 0:
        d //= 2
        s += 1

    for _ in range(k):
        a = random.randint(2, n - 2)
```

```
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue

        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True


def Generate_num(bits):
    while True:
        my_number = random.getrandbits(bits) | (1 << bits - 1) | 1
        if RabinMillerTest(my_number):
            return my_number


def EuclideanAlgorithmExtend(a, b):
    if a == 0:
        return b, 0, 1
    gcd_value, x1, y1 = EuclideanAlgorithmExtend(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd_value, x, y


def MultiplicateInv(e, phi):
    gcd_value, x, _ = EuclideanAlgorithmExtend(e, phi)
    if gcd_value != 1:
        raise Exception('Мультиплікативний обернений не існує.')
    return x % phi



def Gen_Keys(key_size):
    p = Generate_num(key_size // 2)
    q = Generate_num(key_size // 2)
    while p == q:
        q = Generate_num(key_size // 2)
    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    if gcd(e, phi) != 1:
        e = random.randrange(3, phi, 2)
        while gcd(e, phi) != 1:
            e = random.randrange(3, phi, 2)

    d = MultiplicateInv(e, phi)

    return (e, n), (d, n), p, q


def Encryption(public_key, data):
    e, n = public_key
    block_size = (n.bit_length() - 1) // 8
    encrypted_data = []
    for i in range(0, len(data), block_size):
        block = int.from_bytes(data[i:i + block_size], byteorder='big')
        encrypted_block = pow(block, e, n)
        encrypted_data.append(encrypted_block)
```

```
    return encrypted_data


def Decryption(private_key, encrypted_data):
    d, n = private_key
    decrypted_data = bytearray()
    for block in encrypted_data:
        decrypted_block = pow(block, d, n)

decrypted_data.extend(decrypted_block.to_bytes((decrypted_block.bit_length()
+ 7) // 8, byteorder='big'))
    return decrypted_data


def save_encrypted(filename, encrypted_data):
    with open(filename, 'w') as f:
        for item in encrypted_data:
            f.write(f"{item}\n")


def load_encrypted(filename):
    with open(filename, 'r') as f:
        return [int(line.strip()) for line in f]


def Encrypt_File(input_filename, output_filename, public_key):
    with open(input_filename, 'rb') as f:
        file_data = f.read()

    encrypted_data = Encryption(public_key, file_data)
    save_encrypted(output_filename, encrypted_data)

    return input_filename, output_filename



def Decrypt_File(input_filename, output_filename, private_key):
    encrypted_data = load_encrypted(input_filename)
    decrypted_data = Decryption(private_key, encrypted_data)

    with open(output_filename, 'wb') as f:
        f.write(decrypted_data)

    return input_filename, output_filename



def main():
    key_size = int(input("Введіть довжину ключа (має ділитися на 64,
рекомендована 1024): "))
    if key_size < 64 or key_size > 2048 or key_size % 64 != 0:
        print("Довжина ключа має бути від 64 до 2048 біт і ділитися на 64.")
        return

    public_key, private_key, p, q = Gen_Keys(key_size)

    print("\nКомпоненти відкритого ключа:")
    print(f"n (модуль): {public_key[1]}")
    print(f"e (експонента): {public_key[0]}")

    print("\nКомпоненти закритого ключа:")
    print(f"n (модуль): {private_key[1]}")
    print(f"d (закрита експонента): {private_key[0]}")
    print(f"p (перше просте число): {p}")
```

```
    print(f"q (друге просте число): {q}")

    input_filename = input("\nВведіть ім'я файлу для шифрування: ")
    encrypted_filename = input("Введіть ім'я файлу для запису зашифрованої
інформації: ")
    input_file, encrypted_file = Encrypt_File(input_filename,
encrypted_filename, public_key)
    print(f"\nФайл '{input_file}' успішно зашифровано у файл
'{encrypted_file}'.")

    decrypted_filename = input("\nВведіть ім'я файлу для розшифрування
інформації: ")
    encrypted_file, decrypted_file = Decrypt_File(encrypted_filename,
decrypted_filename, private_key)
    print(f"\nФайл '{encrypted_file}' успішно розшифровано у файл
'{decrypted_file}'.")


if __name__ == "__main__":
    main()
```